T.I.P. TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	Oct 21, 2026	**DATE SUBMITTED:**	Oct 21, 2026
NAME:	Antonio, Christine Mae Libo-on, Gian Carlo	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 14.1 Algorithmic Complexity

1. PROCEDURE OUTPUT

	Best Case	Worst Case	Analysis
Shell Sort	O(n log n)	$O(n^2)$	It has a best case time complexity O(n log n) as the elements only have to be compared once. This is the result when the array is already sorted. Its worst case time complexity O(n2) occurs when all the elements are unsorted.
Merge Sort	O(n log n)	O(n log n)	The time complexity for merge sort is O(n log n) for both best case and worst case. The merge sort algorithm divides the array in half and its subarrays in half until it reaches only a single element, and then sorts the elements. It divides and compares the elements.

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

	Algorithm 1 (Shell Sort)	Algorithm 2 (Merge Sort)
Best Case	`Shell sort time complexity for n = 128 (average of 10000 runs):` `Best case  O(n log n) - already sorted: 2568 nanoseconds`	`Merge sort time complexity (average of 10000 runs):` `Best case  O(n log n) - already sorted: 12776 nanoseconds`

Worst Case	`Worst case O(n^2) - interleaved: 8057 nanoseconds`	`Worst case O(n log n) - interleaved: 12923 nanoseconds`
Analysis	The shell sort best case runtime was 2568 ns and has worst case time of 8057 ns.	The best case and worst case runtime are close with best case as 12,776 ns and worst case as 12,923 ns.

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A:

PSEUDOCODE ANALYSIS

Binary search involves halving the array elements for each iteration. It takes less time to reach it's worst time complexity since it shrinks the arrays every iteration. On the other hand, linear search sequentially checks the elements of an array in a 1 to 1 ratio. Meaning, if there are 1000 elements, it will be able to check its worst time complexity after 1000 steps.

TIME AND SPACE COMPLEXITY

The time complexity of the binary search is O(log n) while for linear, it's O(n) which means that binary search is faster than linear search. The same can be said for their space complexity. Since I used the recursive method, my space complexity for binary search is O(log n) and for linear, it's O(n).

Input Size (N)	Elapsed Time for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1,000	Searching Algorithm Comparison (1000 Elements) Target: -1 Algorithm Comparisons Execution Time Linear Search 1000 2300 ns Binary Search 18 100 ns	Searching Algorithm Comparison (1000 Elements) Target: -1 Algorithm Comparisons Execution Time Linear Search 1000 2300 ns Binary Search 18 100 ns
10,000	Searching Algorithm Comparison (10000 Elements) Target: -1 Algorithm Comparisons Execution Time Linear Search 10000 24000 ns Binary Search 26 200 ns	Searching Algorithm Comparison (10000 Elements) Target: -1 Algorithm Comparisons Execution Time Linear Search 10000 24000 ns Binary Search 26 200 ns
100,000	Searching Algorithm Comparison (100000 Elements) Target: -1 Algorithm Comparisons Execution Time Linear Search 100000 210700 ns Binary Search 32 200 ns	Searching Algorithm Comparison (100000 Elements) Target: -1 Algorithm Comparisons Execution Time Linear Search 100000 210700 ns Binary Search 32 200 ns

Analysis:

Linear search takes a significantly longer time to check for the worst case time complexity because it iterates through the entire population of the array, one by one. This means that if there are 1000 or even 100000 elements in an array, a linear search will iterate through them 1000 or 100000 times. On the other hand, binary search halves the entire array every iteration which means that it takes a significantly shorter time to find the worst case compared to linear searching.

Conclusion:

Antonio:
The algorithms that were used in this activity were used in previous lessons. This activity helped to view the time complexity for the algorithms using experimental tools. In the procedure, it demonstrates the runtime differences between the worst case and best case for the sorting algorithms.

Libo-on:

This lesson allowed me to recall and further understand searching algorithms and the importance of optimization and time complexity. A program works so much better when it runs efficiently. I also learned that each sorting and searching algorithm works better when used correctly. For example, for larger samples, we should use binary search since it is much faster in larger samples sizes while for smaller samples, it is better to use linear search. Overall, I think that this activity is a nice refresher since I learned a lot especially in determining time complexities of each sorting and searching algorithms.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Idea generation, code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Help with debugging and answering a few questions regarding code implementation**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.